MECANIX

Workshop Management Platform for Lusophone Africa

Project Plan & Technology Recommendation

Architecture decisions, technology stack, integrations, and sprint plan

Version: 1.0
Date: March 2026
Prepared for: Rau Kurkam
Classification: Confidential – Internal Use Only

Table of Contents

1. Product Overview
2. Architecture Overview
3. Final Technology Stack
4. Build vs Buy Decisions
5. Key Integrations
6. Tenant Management & Billing
7. Key Architectural Decisions
8. Project Structure
9. Sprint Plan (20 Weeks)
10. Monthly Cost at Launch
11. Risk Flags & Recommendations
12. Phase 2 & 3 Roadmap

1. Product Overview

MECANIX is a **cloud-based, offline-first workshop management SaaS** for independent automotive workshops in Lusophone markets – **Angola**, **Mozambique**, and **Brazil**.

The platform unifies vehicle intake, job card management, parts inventory, technician time tracking, invoicing, customer communication, and insurance claim workflows into a single affordable system accessible from any device.

Strategic Vision: MECANIX will be the first serious SaaS native workshop management platform purpose-built for Lusophone Africa and Brazil – combining offline-first architecture, WhatsApp-native communication, a dedicated customer mobile app, and a full insurance assessor portal.

Four Applications, One Backend

App	Target	Users
Workshop App	Web + Mobile	Owners, managers, receptionists, parts manager
Technician App	Mobile	Mechanics
Customer App	Mobile	Vehicle owners
Insurance Portal	Web	Assessors, admins

Target Markets

Market	Currency	Tax	Phase	Key Requirement
Angola	AOA / USD	VAT 14%	Phase 1 (Launch)	Offline-first critical, dual currency, Multicaixa Express
Mozambique	MZN	VAT 17%	Phase 2	M-Pesa integration required
Brazil	BRL	ISS+ICMS	Phase 3	NF-e/NFSe mandatory, LGPD compliance, PIX

Pricing Model

Plan	Target	Angola/MZ Price	Brazil Price	Key Limits
Starter	1 location, up to 3 users	$49/month	R$190/month	500 job cards/month
Professional	1 location, unlimited users	$89/month	R$350/month	Unlimited + reporting
Enterprise	Chains, 2–10 locations	$129/month	R$500/month	Multi-location dashboard
White Label	Dealer groups	Custom	Custom	Full branding + API access
Insurance	Insurance companies	$199/month	R$750/month	Assessor portal + analytics

2. Architecture Overview

```
┌──────────────────────────────────────────────────────────────────────┐
│                          MECANIX Platform                            │
│                                                                      │
│  Workshop App   Technician   Customer App   Insurance Portal         │
│  (Web + Mobile)  App (PWA)   (Mobile PWA)   (Web, Assessor)          │
│                                                                      │
│                        API Layer – NestJS on Railway (Singapore)     │
│                                                                      │
│  Supabase: PostgreSQL (RLS) + Auth + Storage + Realtime              │
│  PowerSync: Offline Sync SDK, 10 client SQLite                       │
│                                                                      │
│  Redis(Upstash) Queue(BullMQ) Cron  Supabase Cloudsql DR             │
│                                                                      │
│  Integrations: WhatsApp Business API | FCM | M-Pesa | Multicaixa     │
└──────────────────────────────────────────────────────────────────────┘
```

Data Flow

1. **Auth:** User logs in → Supabase Auth issues JWT with `tenant_id` claim
2. **Data access:** JWT drives PostgreSQL RLS – every query auto-scoped to tenant
3. **Offline:** PowerSync reads PostgreSQL WAL → syncs to client SQLite → app reads/writes locally
4. **Realtime:** Database changes stream via Supabase Realtime WebSockets to dashboards and mobile apps
5. **Storage:** Photos/PDFs stored in Supabase Storage (S3-compatible), secured by same JWT

3. Final Technology Stack

Layer	Technology	Rationale
Monorepo	pnpm workspaces + Turborepo	Fast builds, shared code across 4 apps
Mobile (3 apps)	React Native + Expo	Cross-platform, TypeScript shared with backend, target hiring pool
Web (3 apps)	Next.js 15 (App Router)	SSR for marketing pages, PWA capabilities, React Server Components
API	NestJS on Fastify	Modules, guards, DI — structured multi-tenant SaaS with 10+ modules
Database	Supabase (PostgreSQL + RLS)	DR + auth + storage + realtime in one. RLS = bulletproof tenant isolation
Auth	Supabase Auth (GoTrue)	Native RLS integration, phone OTP, JWT drives all data isolation
Offline sync	PowerSync	Official Supabase partner, WAL-based sync, production-proven
File storage	Supabase Storage	S3-compatible, image transforms, secured by same JWT
Realtime	Supabase Realtime	WebSocket streaming of DB changes, powers live dashboards
State management	TanStack Query + Zustand	Server state caching + client state. Same pattern across web and mobile
Forms & validation	React Hook Form + Zod	Zod schemas shared between frontend and NestJS backend
i18n (mobile)	i18next + expo-localization	Shared translation files, works with Expo
i18n (web)	next-intl	Native App Router support than react-i18next
PDF generation	@react-pdf/renderer	React-based templating for invoices and workshop branding
Push notifications	Firebase Cloud Messaging	Free, unified iOS + Android
Email	Resend + React Email	Officially API-based delivery
Hosting (web)	Vercel	Zero-config NestJS deploy, edge network
Hosting (API)	Railway	Simple Dockerfile deploy, auto-scaling
CI/CD	GitHub Actions	Lint, type-check, test on every PR
Monitoring	Sentry + Supabase Dashboard	Error tracking + DB monitoring
Testing	Vitest + Playwright + Maestro	Vitest (unit), Playwright (web E2E), Maestro (mobile E2E)

4. Build vs Buy Decisions

Component	Decision	Details
Auth & user management	SUPABASE AUTH	Included in Supabase. Handles JWT, OTP, MFA, RLS integration
Tenant isolation	BUILD	RLS policies + tenants table. Core to product
Plan limits & enforcement	BUILD	Hard-coded checks (one limit) before creating resources.
Subscription invoicing	BUILD	Reuse existing PDF invoice engine for subscription bills.
Payment collection (MCX/M-Pesa)	ZOM WHOLESALE	Generate invoice + WhatsApp + bank transfer + admin confirms.
Payment collection (Brazil)	STRIPE BILLING	Full automation with recurring charges. Phase 3.
Offline sync	POWERSYNC	WAL-based sync, official Supabase partner.
File storage	SUPABASE STORAGE	Included. S3-compatible
Realtime updates	SUPABASE REALTIME	Included. WebSocket streaming.
WhatsApp messaging	META CLOUD API	Official API + template management.
PDF invoices	BUILD	@react-pdf/renderer with workshop branding.
SMS fallback	AFRICA'S TALKING	Better AOA/MZ coverage than Twilio.

Why Not Clerk?

Clerk is excellent for auth + org management, but for MECANIX it creates more problems than it solves:

- **Breaks the RLS chain:** Supabase Auth JWT drives RLS, PowerSync, Realtime, and Storage. Clerk would require a bridge between two auth systems.
- **Double cost:** You'd pay for Clerk while Supabase Auth is already included.
- **Extra latency:** Every authenticated request goes through an additional layer.
- **The work Clerk saves is minimal:** Building login screens + invite flow + role management on Supabase Auth takes ~10 hours.

Why Not Prisma?

Prisma **bypasses RLS**. It connects directly to PostgreSQL, not through Supabase. Supabase client libraries enforce RLS. This defeats the entire multi-tenancy security model. Instead, use the **Supabase JS client** in NestJS, which respects RLS because it uses the user's JWT. For complex queries, use **Kysely** (type-safe SQL query builder).

5. Key Integrations

Integration	Service	When	Cost
WhatsApp Business API	Meta Cloud API or 360dialog (BSP)	Sprint 7	Free tier; ~$15/mo 360dialog + per conversation
Push notifications	Firebase Cloud Messaging	Sprint 7	Free
SMS fallback	Africa's Talking	Sprint 7	~$0.05/SMS
M-Pesa (Mozambique)	Vodacom M-Pesa API	Phase 2	Revenue share
Multicaixa Express (Angola)	EMIS API	Phase 1	Revenue share
PIX (Brazil)	Mercado Pago or Asaas	Phase 3	~1% per transaction
Error monitoring	Sentry	Sprint 0	Free tier
Analytics	PostHog	Sprint 1	Free tier (self-hosted)
Stripe Billing (Brazil)	Stripe	Phase 3	2.9% + $0.30 per transaction

6. Tenant Management & Billing

Tenant Management

Handled natively by the architecture — **no external service needed**:

- Supabase Auth creates users and issues JWTs with `tenant_id` claims
- A trigger creates default workshop profile, settings, and limits
- PostgreSQL RLS protects 186 tables per tenant, by `tenant_id` automatically
- Tenant onboarding: sign up → create tenant record → RLS applies → workshop operational

Subscription Billing — Hybrid Approach

No single billing provider covers all 3 markets. The approach: **build the plan engine, outsource payment collection where possible.**

Custom Billing Module (Week 20)

A lightweight module that handles:

- **Plan logic:** Trains with clear limits (one job card, max users, features)
- **Usage tracking:** Count active jobs, users, storage per tenant
- **Subscription invoicing:** Monthly invoice PDF generation (reuse existing engine)
- **Limit enforcement:** NestJS guard checks plan limits before creating resources
- **Grace period:** 7 days past due + suspend on 30 days + archive
- **WhatsApp reminders:** Automated payment reminders at 7, 3, and 1 day before due

Payment Collection by Market

Market	Method	Automation Level
Angola	Bank transfer + Multicaixa Express	Semi-manual: invoice + customer pays + admin confirms
Mozambique	M-Pesa + bank transfer	Semi-manual: M-Pesa API collection
Brazil (Phase 3)	Stripe Billing (card + PIX)	Fully automated: recurring charges, dunning, portal

7. Key Architectural Decisions

1. **Supabase is the backbone** — not just a database. Auth, storage, realtime, and RLS all flow from a single JWT. Do not break this chain with external auth (Clerk) or external CRM (Pipedrive).
2. **No Prisma** — use the Supabase JS client, which respects RLS. Prisma bypasses RLS and defeats the entire security model. For complex queries, use Kysely or raw SQL via Supabase's `rpc()` function.
3. **No Clerk** — paying for two auth systems while maintaining a bridge adds cost and complexity. Supabase Auth does everything needed and integrates natively with RLS, PowerSync, Realtime, and Storage.
4. **Billing is hybrid** — build a simple plan/limits engine. Angola and Mozambique run on bank transfers and manual confirmation. Automate with Stripe only when entering Brazil.
5. **Offline-first is non-negotiable** — PowerSync from day 1. Every mobile feature must work without internet. This is a market requirement in Angola and Mozambique.
6. **Money as integers** — all amounts stored in centis/cents/avos. No floating point. No exceptions. Rebuilding this is extremely painful.
7. **Hire from day 1** — no hands-on coding, scope, or PM, all 8h, are AI-formatting (dates, numbers, currency) locale-aware via the Intl API.

8. Project Structure

```
mecanix/
├── pnpm-workspace.yaml          # pnpm workspace root
├── turbo.yaml                   # Turborepo config
├── .env.shared                  # CLOB (Clerk, Sentry, keys)
│
├── packages/
│   ├── @mecanix/                # Shared types, zod schemas, constants
│   ├── @mecanix/                # Algorithms, seed data, RLS policies
│   ├── @mecanix/                # Shared UI components (Tamagui)
│   │
│   ├── apps/
│   ├── api/                     # NestJS hosted (Cloud Run middle)
│   ├── web-workshop/            # Next.js workshop dashboard
│   ├── web-insurance/           # Next.js insurance portal
│   ├── mobile-workshop/         # Expo app (workshop staff)
│   ├── mobile-technician/       # Expo app (technicians)
│   └── mobile-customer/         # Expo app (vehicle owners)
│
│   Tooling/
│   ├── eslint-config/           # Shared ESLint config
│   ├── prettier-config/         # Shared Prettier config
│   └── tsconfig/                # Shared TSconfig base
│
│   docs/
│   └── specs/                   # Specification documents
```

9. Sprint Plan — 20 Weeks

Team: 7.2 developers + Claude Code as force multiplier
Total hours: ~1,440 hours
Total sprints: 10 (Sprint 0–9)

Sprint 0 — Stack Ramp-Up

Weeks 1 0.5 – 2 hours

Supabase quickstart, NestJS scaffold, Expo app, PowerSync offline test, Next.js setup, deploy everything Railway + Vercel, Gitmal Actions CI, i18n setup. **Output:** Throwaway proof of stack, where all pieces connect.

Sprint 1 — Foundation & Tenant System

Weeks 2 8 – 2 40 hours

Production monorepo, database schema (tenants, users, customers, vehicles) with RLS, TenantGuard, auth flow, tenant onboarding, register/login screens (mobile + web), PowerSync production sync rules, i18n configuration, deploy pipeline.

Sprint 2 — Customer & Vehicle Management

Weeks 3 8 – 2 40 hours

Full CRUD for customers and vehicles, search (name/phone/plate/VIN), vehicle service history, photo uploads, customer facilitated pages (web), mobile customer search + vehicle registration with camera, offline creation/sync verification.

Sprint 3 — Job Card System (Core Product)

Weeks 4 8 – 2 40 hours (highest sprint)

Job card CRUD with auto-numbering, 8-status engine workflow with validation, labour lines, parts lines, job cards settings (insurance-toggle, digital auth, parts issuing mode, tenable toggle), technician profiles, job card kanban/list pages (web + mobile), offline job cards.

Sprint 4 — Technician Time Logging

Weeks 5 8 – 2 40 hours

Dedicated technician flow app, one-tap start/stop timers, clock in/out, job interaction (notes, photos, flags), productivity dashboard, live floor view (manager view), daily timesheet report, offline time tracking, US review for ground hands.

Sprint 5 — Parts, Inventory & Suppliers

Weeks 6 8 – 2 40 hours

Parts catalogue, stock management with low stock alerts, parts issuing workflow (auto on manual), service groups, inventory adjustments, vendor management, purchase orders, bills, vendor credits, expense tracking with receipt scanning.

Sprint 6 — Invoicing & Payments

Weeks 7 8 – 2 40 hours

Auto-generate invoices from job cards, configurable tax engine (15% IVA/17%), PDF generation with branding, payment recording (cash/transfer/card/M-Pesa/PIX), partial payments, split billing (customer + insurer), credit notes, financial dashboard widgets, accounts receivable aging.

Sprint 7 — WhatsApp + Customer App

Weeks 8 8 – 2 40 hours

WhatsApp Business API integration, message templates for all triggers, notification on status changes, SMS fallback, Customer App: phone OTP onboarding, my vehicles, live job tracking with push notifications, digital quote approval, invoice view, service history, Firebase Cloud Messaging setup.

Sprint 8 — Insurance Module

Weeks 9 8 – 2 40 hours

Insurance claim, 7-stage workflow, structured estimate submission, reviews with partial (claims queue), line by line estimate review, sale card comparison), mandatory GPS-tagged damage photos, Insurance/Assessor (cross-tenant read access), fraud detection (duplicate claims, anomaly flagging), report monitoring, restoration rate insurance job card enhancements.

Sprint 9 — Dashboard, Reports & Beta Launch

Owner dashboard (financial pulse, revenue, receivables/payables, income vs expenses), 11 report endpoints with PDF export, service reminder engine, workshop settings (logo, tax, branding), offline sync hardening, performance testing (50 concurrent users), 186 review, bug bash, beta onboarding 2–5 workshops, Sentry monitoring, quick start documentation.

10. Monthly Cost at Launch

Service	Plan	Monthly Cost
Supabase	Pro	$25
Railway (API)	Trial / Pro	$0 – $20
Vercel	Pro	$20
Sentry	Free tier	$0
PostHog	Free tier	$0
Claude Code	Max	$100
Total		**~$165–$265/month**

Comparison: The same stack on AWS: ECS + RDS + S3 + custom auth = custom sync, earned would cost ~$1500/month and require 0.5 additional weeks of DevOps setup.

11. Risk Flags & Recommendations

Risk	Impact	Mitigation
PowerSync is the biggest technical risk	High	Invest Sprint 0 in a proof-of-concept. Test offline mode/edit/sync cycle before writing production code.
WhatsApp Business API approval takes 2–6 weeks	Medium	Apply for Meta Business account immediately, even before coding starts. Use 360dialog BSP if direct approval is slow.
Workshop adoption resistance	High	Simple onboarding (<10 min), free setup for first 20 customers, champion program.
Multicaixa/M-Pesa integration complexity	High	Offline-first architecture with PowerSync. Conflict resolution: server authoritative for status, merge for time entries, append-only for photos.
Spirit S Cycle Cost/Day exceeds revenue	Medium	This is the heaviest sprint (~8h). Allow 1–2 days spillover into Sprint 4. Job card system must be solid before building on top.
No billing provider covers Angola/Mozambique	Medium	Hybrid approach: semi-manual billing (invoice + bank transfer + admin confirmation) for AOA/MZ. Stripe for Brazil.
Insurance company relationships to adopt	High	Start with 1–2 insurer partnerships. Prove ROI with pilot data. Offer free onboarding.

Scope deferred priority: — parts catalog, quote flow (without inventory embedding), vendor credits, service groups, Fraud detection, customer app payments, expense tracking, **Never defer:** job card lifecycle, offline sync, WhatsApp notifications, invoice generation, basic insurance workflow.

12. Phase 2 & 3 Roadmap

Phase 2 — Months 5–12

Area	Features
Inventory	Tecdoc parts catalogue integration, lyre management module
Payments	M-Pesa integration (Mozambique), Multicaixa Express (Angola)
Finance	Accounting software integration (Zoho Books, Cogest Primavera)
Scheduling	Advanced bay management, online appointment booking, shop calendars
Operations	Multi-location support, gamification based technician incentives
Customer	Workshop ratings, service reminders, promotional campaigns, CRM module
Insurance	Advanced analytics, bulk payments, API for insurer CMS
Platform	OAUTH2 Sign Integration, expanded reporting

Phase 3 — Months 13–24

Area	Features
Book entry	NF-e/NFSE/ Fiscal integration, pt-BR localization, Scoop-competitor insurance
Financial maturity	Full accounting (journals, chart of accounts), payroll, membership/loyalty programs
Billing automation	Stripe Billing for Brazilian market (recurring charges, PIX, dunning)
Platform growth	White-label packaging, reseller portal, enterprise API access
AI features	Service interval prediction, estimate anomaly detection, repair time estimation
Scale	Evaluate migration to self-managed infrastructure if cost-beneficial